TripAdvisor Information Retrieval Project

Information Retrieval and NLP

Maxim GROSSMANN
Geoffroy-Junior GANKOUE-DZON

March 3, 2026

Contents

1 Introduction **4**

1.1 Experimental Pipeline 4

2 Datasets and Preprocessing **5**

2.1 Datasets 5

2.2 Metadata Cleaning 5

2.3 Review Preprocessing 5

2.4 Separation of Modeling and Evaluation Data 6

3 Selection Methods **6**

3.1 Selection Framework and Caching 6

3.2 Selection Strategy Families 7

3.2.1 A_baseline_all — Full Concatenation 7

3.2.2 B — Review-Level Filtering 7

3.2.3 C — Sentence-Level Filtering 7

3.2.4 D — TF-IDF Word Budget 7

3.2.5 E — Semantic and Generative Methods 7

4 Evaluation Metrics **8**

4.1 Relevance Levels 8

4.2 Precision@k, Recall@k, Hit@k 8

4.3 MRR 8

4.4 nDCG@k 8

4.5 Ranking Error (first relevant rank) 8

5 First Evaluation: Screening Selection Methods with BM25 **9**

5.1 Setup 9

5.2 Top-5 Methods 9

5.3 Deeper Analysis: Why Some Selection Methods Work Better 9

5.3.1 TF-IDF word budgets: why they perform best under BM25 . . . 9

5.3.2 Median-length review selection: why it is a strong heuristic 10

5.3.3 Rating-balanced sampling: why it did not consistently win 11

5.3.4 Sentence-level TF-IDF: competitive but weaker than word budgets under BM25 11

5.3.5 Dense extractive and LLM summarization: strong ideas, not BM25-optimized 12

5.3.6 Overall takeaway for selection design 12

6 Ranking Model Comparison (917 Queries) **13**
6.1 Key Trends 13
6.2 Best Overall Configurations 14

7 Discussion **14**
7.1 Why lexical baselines remain strong 14
7.2 Why dense bi-encoders help (and why they do not always win) 15
7.3 Why hybrid retrieval is consistently robust 15
7.4 Why L2 relevance is difficult and what the metrics reveal 16
7.5 Why reranking improves strict relevance (L2) 16
7.6 Winner: which model should be considered "best"? 17
7.7 What could be improved next (model side) 17

8 Conclusion **18**

A Appendix: Results Tables **18**
A.1 BM25 Screening (All Selection Methods) — Key Metrics 19
A.2 Top-5 Selection Methods × Models (35 Runs) — Key Metrics 19

1 Introduction

The objective of this project is to design and evaluate an Information Retrieval (IR) system capable of recommending similar places using user-generated reviews as the only source of information. Given a query text, the system returns a ranked list of places whose aggregated review text is most similar to the query. This is a place-level retrieval task: each candidate corresponds to a place (not an individual review), and the ranking is computed by comparing the query text to one document representation per place.

In our intended use case, a query can be a single review written by a user. However, for the offline evaluation reported in this work, we used **place-level documents as queries** (constructed from reviews) to obtain a large number of stable queries and to keep the evaluation fully consistent with metadata-derived ground truth.

A key challenge is representation. Each place is associated with many reviews of variable length, quality, and focus. Naïvely concatenating all reviews produces long and noisy documents, which can harm both lexical and semantic retrieval. Therefore, the core experimental question of the project is:

> *Which document construction strategy (review selection/compression) and which retrieval model yield the best ranking performance for review-driven place retrieval?*

1.1 Experimental Pipeline

We followed a structured pipeline:

1. **Data import:** load `reviews83325.csv` and `Tripadvisor.csv`.
2. **Preprocessing:** clean and normalize review text; clean metadata.
3. **Document construction:** implement 18 selection strategies to build one text document per place.
4. **Evaluation framework:** implement L1/L2 relevance definitions from metadata and compute ranking metrics.
5. **Stage 1 (screening):** evaluate all selection methods with BM25 and keep the top 5.
6. **Stage 2 (model comparison):** evaluate the top-5 selection methods against 7 model configurations grouped into 5 model families, resulting in 35 runs.

The goal is not to validate a theoretical hypothesis, but to perform a systematic comparison in order to identify the most effective configuration for this task.

2 Datasets and Preprocessing

2.1 Datasets

We worked with the following TripAdvisor datasets:

- **reviews83325.csv**: 340,385 reviews (raw file; one row per review).
- **Tripadvisor.csv**: place-level metadata (IDs and categories).
- **restaurantType.csv**, **cuisine.csv**, **dietary_restrictions.csv**.
- **AttractionSubCategorie.csv**, **AttractionSubType.csv**.

Only `reviews83325.csv` and `Tripadvisor.csv` are required for the ranking experiments, but the additional files were inspected to understand the category structure and to inform the design of L2 relevance.

2.2 Metadata Cleaning

In `Tripadvisor.csv`, we removed 14 columns that were entirely empty. This simplifies downstream processing and ensures the evaluation table contains only meaningful fields.

2.3 Review Preprocessing

From the reviews dataset, we applied minimal but sufficient preprocessing to preserve meaning while ensuring stable tokenization:

- **Language filtering:** we kept only English reviews, resulting in 153,071 reviews (about 45% of the raw file).
- **HTML normalization:** HTML artifacts were limited, so we applied lightweight cleanup (unescape entities + normalize whitespace).
- **Rare character cleanup:** unusual characters were rare (<0.1% of tokens). We removed problematic characters (NBSP, zero-width, control chars, replacement char) to prevent tokenization issues without altering semantics.
- **Accent normalization:** accents were normalized to avoid inconsistent lexical matching.

After preprocessing, we built the working review-level dataframe:

```
df_models = reviews_en[["id", "idplace", "text", "note"]].copy()
df_models.columns = ["review_id", "place_id", "review_text", "score"]
```

This produced:

- **153,071 reviews**
- **1,835 unique places**
- **0 missing texts and 0 empty texts**

2.4 Separation of Modeling and Evaluation Data

We separated the pipeline into two datasets with different roles:

- **df_models** (review-level): `(review_id, place_id, review_text, score)`. This is the input for all selection methods and remains independent of any retrieval model.
- **df_eval** (place-level): the full metadata table (cleaned `Tripadvisor.csv`). It is intentionally kept free of review text to serve as a stable ground-truth source for relevance definitions.

We evaluate relevance at two levels:

- **L1:** broad similarity (same `typeR`).
- **L2:** stricter similarity based on subcategory overlap (e.g., attraction subtype/category, restaurant type/cuisine, hotel price range).

Both dataframes were stored as **Parquet** files to ensure reproducibility and fast I/O.

3 Selection Methods

Each selection method transforms `df_models` into a place-level corpus with one row per place:

$$\texttt{place_id} \rightarrow \texttt{text}$$

This corpus is then indexed by retrieval models.

3.1 Selection Framework and Caching

To ensure consistency and efficient experimentation, we implemented a modular framework:

- **base.py**: enforces a strict output schema (at least `place_id` and `text`) and computes diagnostics (e.g., `n_chars`).

- **registry.py**: defines the full list of methods and hyperparameters in a single place for systematic iteration.
- **cache.py**: caches corpora using a hash of (method, parameters), preventing recomputation and ensuring reproducibility.

All corpora are built once and cached using a build script that loads `df_models.parquet`, iterates over the registry, calls each method's `build()` function, enforces the output schema, and stores the resulting place corpus as Parquet.

3.2 Selection Strategy Families

We implemented several complementary method families, each targeting a different trade-off between coverage and noise reduction.

3.2.1 A_baseline_all — Full Concatenation

Concatenate all reviews for a place into a single document. **Purpose:** baseline reference. **Limitation:** long, redundant, and noisy.

3.2.2 B — Review-Level Filtering

B_longest_k Keep the k longest reviews per place.

B_medium_k Keep the k reviews closest to the median length (per place).

B_balanced_k Select reviews across rating buckets (low/mid/high).

3.2.3 C — Sentence-Level Filtering

C_best_sent_n Split into sentences, score with TF-IDF, and keep the top n sentences.

3.2.4 D — TF-IDF Word Budget

D_tfidf_budget (80/120/160) Keep the top TF-IDF words under a fixed budget to maximize discriminative vocabulary.

3.2.5 E — Semantic and Generative Methods

E_dense_extractive Use sentence embeddings + KMeans to select representative sentences (semantic diversity).

E_llm_ollama (Qwen) Generate a compact retrieval-oriented summary per place (limited to 50 places due to runtime).

4 Evaluation Metrics

For each query, the system ranks candidate places and we compute metrics based on relevance derived from metadata.

4.1 Relevance Levels

- **L1 relevance:** same broad place type (`typeR`).
- **L2 relevance:** overlap in finer-grained categories extracted from metadata fields.

For L2, categories are extracted from subtype fields (e.g., attraction subcategory/subtype, restaurant type/cuisine, hotel price range), and two places are L2-relevant if they share at least one category. In the metadata, `typeR` encodes the coarse place type (e.g., restaurants, hotels, attractions), and is used to define L1 relevance.

4.2 Precision@k, Recall@k, Hit@k

Precision@k and Recall@k measure the proportion and coverage of relevant items in the top-k results. Hit@k indicates whether at least one relevant result appears in the top-k. We report $k \in \{1, 5, 10, 20\}$.

4.3 MRR

Mean Reciprocal Rank measures how early the first relevant item appears; higher is better.

4.4 nDCG@k

nDCG evaluates ranking quality with position-based discounting and graded gains. In our implementation, L1 matches have gain 1 and L2 matches gain 2.

4.5 Ranking Error (first relevant rank)

Ranking error is the 0-based rank of the first relevant item:

- **ranking_error_L1:** first rank where an L1-relevant candidate appears.
- **ranking_error_L2:** first rank where an L2-relevant candidate appears (or the ranking length if none appears).

Lower is better.

5 First Evaluation: Screening Selection Methods with BM25

Before comparing ranking models, we screened all **18 selection methods** using BM25 to identify the strongest document construction strategies under a lexical baseline.

5.1 Setup

For each selection method, we loaded the cached place corpus, filtered to places present in `df_eval`, split places into **50% train / 50% test** (seed=42), fit BM25 on the test split (candidate index), and issued queries from the train split (excluding the query place ID). We evaluated at most **200 queries** per method (MAX_QUERIES=200).

5.2 Top-5 Methods

D_tfidf_80, D_tfidf_160, D_tfidf_120, B_medium_k10, B_medium_k20

Table 1: Top 5 selection methods under BM25 (200 queries).

Method	MRR_L1	P@1_L1	nDCG@10	ranking_error_L1
D_tfidf_80	0.944	0.920	0.849	0.365
D_tfidf_160	0.936	0.900	0.846	0.485
D_tfidf_120	0.930	0.890	0.839	0.335
B_medium_k10	0.925	0.885	0.837	0.540
B_medium_k20	0.912	0.860	0.830	0.470

5.3 Deeper Analysis: Why Some Selection Methods Work Better

The BM25 screening step is intentionally "lexical-first": BM25 only rewards **exact term overlap** (with IDF weighting and length normalization). As a result, selection methods are primarily judged by how well they (i) preserve **discriminative vocabulary** and (ii) remove **generic, high-frequency boilerplate** that creates false matches. In other words, the dominant factor is the **signal-to-noise ratio** of the final place document.

5.3.1 TF-IDF word budgets: why they perform best under BM25

The TF-IDF budget methods (D_tfidf_80/120/160) consistently rank at the top because they act as an aggressive **lexical denoiser**.

- **Boilerplate suppression:** Many reviews contain universal phrases ("great place", "friendly staff", "highly recommend") that appear across thousands of places. These phrases have weak discriminative power and can inflate BM25 scores for irrelevant candidates. TF-IDF naturally down-weights such terms.

- **Lexical "fingerprints":** After filtering, the remaining terms are more likely to be category-specific and place-specific (cuisine names, activity types, location cues, distinctive adjectives). BM25 directly benefits because these terms carry high IDF and create sharp matches.

- **Reduced length variance:** A fixed budget standardizes document size. Since BM25 includes length normalization, reducing document-length variance makes scores more comparable across places and improves ranking stability.

Why 80 vs 120 vs 160. Smaller budgets maximize precision by keeping only the strongest discriminative terms, while larger budgets improve coverage but may reintroduce weaker tokens. The close performance of these budgets suggests that many places have a strong "core" of discriminative vocabulary, and the optimal budget depends on the precision–coverage trade-off.

How to improve TF-IDF budgets further.

- **Align preprocessing with retrieval:** apply the same tokenization and stopword policy used by BM25 when computing TF-IDF, ensuring the selected tokens are those the retriever actually uses.

- **Diversity-aware selection:** prevent near-duplicates (singular/plural variants, repeated stems) from consuming the budget. A simple constraint (e.g., one token per stem/lemma or penalizing highly similar tokens) can increase coverage without increasing noise.

- **Anchor multiword expressions:** many discriminative concepts are phrases (e.g., "street food", "wine bar"). Extending the budget to include bigrams/trigrams can strengthen lexical fingerprints.

5.3.2 Median-length review selection: why it is a strong heuristic

The B_medium_k methods perform well because they implicitly select reviews with **high information density**. In review corpora, extremes are often problematic:

- very short reviews are frequently pure sentiment with little content (low signal),

- very long reviews can include personal narratives, digressions, and repeated context (higher noise),

- medium-length reviews tend to describe concrete aspects (food, service, location, activities) with limited redundancy.

This also explains why B_longest_k does not dominate: long reviews may be rich, but they can introduce additional non-discriminative language that BM25 matches spuriously.

How to improve the "medium" family.

- **Quality scoring instead of length-only:** rank candidate reviews by a combined score (length-in-range + lexical diversity + presence of concrete nouns/adjectives) rather than distance-to-median alone.
- **Aspect coverage constraint:** avoid selecting multiple medium-length reviews that repeat the same aspect. A simple heuristic is to prefer reviews that add new vocabulary (or new aspect keywords) relative to already selected ones.

5.3.3 Rating-balanced sampling: why it did not consistently win

The goal of B_balanced_k is reasonable: sample across low/mid/high ratings so that the representation includes both positive and negative cues (pros/cons). However, lexical retrieval can limit the benefits:

- **Sentiment is often generic:** negative reviews can be vague ("too expensive", "not worth it") and add little discriminative information while still increasing noise.
- **Forced inclusion can dilute signal:** enforcing balance may include weaker reviews simply to satisfy the bucket constraint, lowering the overall density of useful terms.

How to improve balanced sampling.

- **Two-stage selection:** first filter reviews by a quality score (as above), then apply balancing within the remaining high-quality set.
- **Bucket weighting instead of strict balancing:** use a soft constraint (e.g., prefer diversity but do not force low-signal reviews into the document).

5.3.4 Sentence-level TF-IDF: competitive but weaker than word budgets under BM25

C_best_sent_n extracts informative sentences, which is a good denoising idea. It can still underperform TF-IDF budgets for BM25 because the output remains **sentence-shaped**: even high-quality sentences contain many non-discriminative tokens that "consume space" without adding lexical specificity. In contrast, a word budget is an explicitly BM25-optimized representation.

How to improve sentence extraction.

- **Boilerplate filtering:** remove or down-weight common sentence templates before TF-IDF scoring.
- **Diversity-aware selection:** penalize selecting sentences that are lexically or semantically redundant, so the final set covers multiple aspects (service, food, atmosphere, location).

5.3.5 Dense extractive and LLM summarization: strong ideas, not BM25-optimized

Dense extractive summarization and LLM summarization aim to preserve semantic diversity and coherence. However, BM25 does not reward semantic similarity; it rewards **exact lexical overlap**. Consequently:

- dense selection may choose sentences that are semantically representative but not lexically distinctive,
- LLM summaries may paraphrase and replace keywords with synonyms, reducing exact overlap even when meaning is preserved.

This does not imply these methods are weak overall. Instead, they may be better matched to **dense** or **hybrid** retrievers than to pure lexical retrieval.

How to make these methods more compatible with lexical retrieval.

- **Keyword anchoring:** enforce the inclusion of a small set of high-TF-IDF keywords or named entities in the final text.
- **Hybrid documents:** concatenate a coherent summary (dense/LLM) with a short TF-IDF keyword list (e.g., 30–50 tokens). This preserves readability while guaranteeing lexical anchors for BM25.

5.3.6 Overall takeaway for selection design

Across methods, the screening suggests that the best place documents are those that:

- remove boilerplate and redundancy,
- preserve discriminative, category-specific vocabulary,
- reduce length variance across places,
- increase coverage of multiple aspects without adding low-signal text.

These principles explain why TF-IDF budgets and median-length selection were retained in the top-5 methods for the full model-comparison stage.

6 Ranking Model Comparison (917 Queries)

We evaluated the top-5 selection methods against 7 model configurations (grouped into 5 model families), resulting in 35 runs. This stage used **917 queries** (MAX_QUERIES=None).

Table 2: Top 5 configurations on the 917-query evaluation, ranked by nDCG@10 (higher is better).

Selection Method	Model	nDCG@10	MRR_L1	P@1_L1	MRR_L2	ranking_error_L2
B_medium_k20	dense_mpnet	0.8393	0.9220	0.8822	0.3543	507.3402
B_medium_k10	dense_mpnet	0.8388	0.9215	0.8811	0.3543	507.3391
D_tfidf_160	hybrid_alpha0.5	0.8348	0.9219	0.8790	0.3702	505.6619
D_tfidf_120	hybrid_alpha0.5	0.8332	0.9256	0.8877	0.3714	505.4133
D_tfidf_80	hybrid_alpha0.5	0.8320	0.9213	0.8757	0.3676	505.6772

Table 3: Best configurations for strict L2 matching (lowest ranking_error_L2; lower is better).

Selection Method	Model	ranking_error_L2	MRR_L2
D_tfidf_80	dense_rerank_top50	28.62	0.3641
D_tfidf_120	dense_rerank_top50	28.69	0.3622
D_tfidf_160	dense_rerank_top50	28.76	0.3571

Table 3 highlights that reranking substantially improves strict L2 matching, even when it does not maximize nDCG@10, illustrating the trade-off between overall top-k quality and fine-grained relevance.

6.1 Key Trends

L1 performance is strong across most models: many configurations achieve MRR_L1 around 0.92 and P@1_L1 close to 0.88, showing that broad-type similarity is recovered reliably.

L2 is substantially harder: many configurations yield large `ranking_error_L2` values (around 505–507), meaning strict fine-grained matches are often not ranked early. The main exception is **dense reranking**, which reduces `ranking_error_L2` to around 28–29 (first strict match appears much earlier).

6.2 Best Overall Configurations

Table 2 shows that the highest nDCG@10 is achieved by **dense MPNet** combined with **medium-length review selection** (B_medium_k10/k20). This suggests that dense encoders benefit from coherent, sentence-like representations.

Hybrid retrieval remains highly competitive and dominates among TF-IDF budget corpora, indicating that lexical and semantic signals are complementary: dense retrieval is strong when representations preserve fluent context, while hybrid retrieval is especially effective when representations are compact and lexically discriminative.

7 Discussion

This discussion focuses on **model behavior**: why certain ranking models perform better than others under our evaluation protocol, and what these results imply about review-driven place retrieval. Beyond reporting metrics, the goal is to explain the observed trends using well-known IR principles (lexical discrimination, length normalization) and neural retrieval theory (semantic generalization, embedding limitations, interaction models).

7.1 Why lexical baselines remain strong

A central result is that lexical baselines (BM25 and TF-IDF) remain highly competitive, especially when place representations are compact and discriminative (e.g., TF-IDF budgets or carefully selected reviews). This is expected for two main reasons:

- **Term discrimination and IDF effects:** Many place attributes are expressed through characteristic keywords (cuisine types, activity names, neighborhood/location cues, "family-friendly", "museum", etc.). In classical IR, the inverse document frequency (IDF) amplifies terms that appear in fewer documents, making them powerful signals. When selection methods preserve these terms, BM25/TF-IDF can retrieve relevant places through exact matches.

- **Boilerplate noise in reviews:** Review datasets contain extensive shared phrasing (e.g., "great place", "friendly staff", "highly recommend"). These terms do not separate places well, but they can still create spurious matches if not filtered. Selection methods that reduce boilerplate increase the relative weight of discriminative terms, directly improving lexical retrieval.

BM25 often matches or slightly outperforms TF-IDF because it incorporates **document-length normalization** and **term-frequency saturation**. Length normalization reduces the advantage of long documents (which otherwise match more terms by chance), and saturation prevents repeated terms from dominating the score. In our setting, where

document lengths differ substantially across places (depending on review count and selection method), these design choices are particularly beneficial.

7.2 Why dense bi-encoders help (and why they do not always win)

Dense retrieval models (Sentence-BERT variants) encode text into continuous vectors, enabling matching beyond exact words. This is useful because similar experiences can be described with different vocabulary (paraphrases, stylistic variations, indirect references). Dense models therefore often improve **semantic recall**, particularly when lexical overlap is weak.

However, dense bi-encoders also have known limitations in review-driven retrieval:

- **Generic sentiment collapse:** Reviews frequently contain shared sentiment and generic descriptors. When representations are not sufficiently specific, embeddings can become overly similar across many places, reducing separability. This can yield good coarse retrieval (L1) but weaker fine-grained ranking.
- **Dependence on fluent context:** Bi-encoders tend to encode meaning best from coherent, sentence-level text. Extremely compressed representations (e.g., keyword lists) reduce syntax and discourse structure, which can weaken semantic modeling. This is consistent with the strong performance of dense MPNet combined with medium-length review selection: medium-length reviews preserve natural language structure while still limiting noise.

In summary, dense models are strong when the representation contains enough coherent context to express distinctive aspects of a place. They are less reliable when the input is dominated by generic sentiment or when the representation removes too much linguistic structure.

7.3 Why hybrid retrieval is consistently robust

Hybrid retrieval combines two complementary mechanisms:

- **Lexical anchors:** exact matches on discriminative terms (BM25 component).
- **Semantic generalization:** similarity under paraphrase or vocabulary mismatch (dense component).

The theoretical advantage is that hybrid retrieval reduces the failure modes of each component. If a query contains a highly discriminative keyword, BM25 can strongly

anchor the match. If the query uses different phrasing (synonyms, paraphrases), dense similarity can still recover relevant candidates.

In practice, hybrid retrieval tends to be especially effective when place representations are both **compact** and **content-rich**: compact enough to suppress boilerplate, but rich enough to preserve meaningful context. This explains why hybrid configurations on TF-IDF budgets remain near the top of the rankings and provide stable top-k quality, even when they are not strictly the absolute best on a single metric.

7.4 Why L2 relevance is difficult and what the metrics reveal

Our evaluation distinguishes coarse relevance (L1) from strict relevance (L2). L2 requires overlap in fine-grained metadata categories, but fine-grained category evidence is often **sparse in text**:

- The subcategory may not be explicitly mentioned in the review.
- It may be expressed through indirect cues or synonyms.
- Reviews may focus on generic experience rather than category-specific features.

As a consequence, many models achieve strong L1 performance (retrieving the correct broad type is relatively easy), while strict L2 matches are often not ranked early. This is exactly what large `ranking_error_L2` values indicate: even if L2-relevant items exist, they are not consistently surfaced near the top without stronger interaction modeling.

This suggests that, in a purely text-based setting, L2 similarity is partly limited by **signal availability** in reviews and partly by model sensitivity to subtle cues. It also motivates using multi-stage retrieval when strict matching is required.

7.5 Why reranking improves strict relevance (L2)

Dense reranking introduces a cross-encoder that scores query–candidate pairs jointly. This differs fundamentally from BM25 and bi-encoders:

- BM25 uses bag-of-words overlap with weighting.
- Bi-encoders compare two independently computed embeddings.
- Cross-encoders model **token-level interactions** across query and candidate.

Because strict L2 relevance often depends on subtle evidence (specific phrases, attribute mentions, fine distinctions), interaction-based scoring is well-suited to correcting large mistakes. This explains why reranking can drastically reduce `ranking_error_L2`:

it can detect the presence of fine-grained cues that are diluted by bag-of-words scoring or averaged embeddings.

The limitation is computational cost: reranking requires scoring many query–candidate pairs (top-50 in our implementation). Therefore, reranking is typically best deployed as a second stage (re-rank a small candidate set) rather than as a full-corpus retriever.

7.6 Winner: which model should be considered "best"?

There is no single model that dominates every objective simultaneously (best L1, best L2, best nDCG, fastest runtime). We therefore choose the winner based on the most realistic objective for a recommendation-style IR system: **strong top-k quality with reasonable computational cost**.

Winner by pure ranking quality (nDCG@10). If we select the winner strictly by nDCG@10, the best-performing configuration in our 917-query comparison is:

Dense MPNet + B_medium_k20

This indicates that a strong dense encoder combined with coherent, medium-length review representations can produce the most effective top-10 ranking.

Winner by robustness and deployability. If the goal is a robust system that remains strong across representations and is computationally efficient, then **hybrid retrieval** is the best default. Its performance is consistently near the top, and it avoids the heavy cost of reranking while reducing both lexical-only and dense-only failure modes.

When reranking is the winner. If the objective is strict fine-grained relevance (L2) and compute cost is acceptable, then **dense reranking** becomes the best choice because it substantially improves the position of the first L2-relevant match. A practical compromise is a two-stage pipeline: hybrid or dense retrieval for candidate generation, followed by reranking on the top-N candidates.

7.7 What could be improved next (model side)

The results suggest several concrete improvements:

- **Tune hybrid weight α:** instead of fixing $\alpha = 0.5$, evaluate a grid (e.g., 0.2/0.5/0.8) or tune per place type (`typeR`), since different categories may benefit differently from lexical anchors vs semantic similarity.
- **Type-aware retrieval:** learn separate configurations per `typeR` (hotels vs restaurants vs attractions), as their review language and discriminative cues differ.

- **Default two-stage design:** retrieve with a fast model (hybrid or dense) and apply reranking only on the top-N candidates when the user requests stricter similarity or when ranking confidence is low.
- **Keyword anchoring for semantic representations:** for dense/LLM-derived place documents, concatenate a short TF-IDF keyword list to preserve lexical anchors, improving both hybrid and lexical components.

Overall, these findings suggest that strong retrieval in this setting is achieved by combining lexical discrimination with semantic generalization, and by using interaction-based reranking when fine-grained relevance must be optimized.

8 Conclusion

We built a complete review-driven place retrieval pipeline and evaluated how document construction and model choice affect ranking performance.

Key conclusions:

- **Document construction strongly impacts retrieval quality.** TF-IDF budgets and medium-length selection produce high-signal representations.
- **Best nDCG@10 in the 917-query evaluation is achieved by dense MPNet with medium-length review selection** (B_medium_k20).
- **Hybrid retrieval is a strong practical default** thanks to stable top-k quality and lower cost than cross-encoder reranking.
- **Dense reranking is most effective for strict L2 relevance**, but introduces significant computational overhead.

Overall, effective IR for reviews requires both careful representation design and an appropriate ranking strategy depending on whether the target is coarse (L1) or fine-grained (L2) similarity.

A Appendix: Results Tables

The tables below report a compact subset of key metrics for readability. For the full metric set (all cutoffs and all metrics), see the CSV files in the `results/` folder:

- `results/bm25_selection_methods_full_metrics.csv`
- `results/models_top5_selection_methods.csv`

A.1 BM25 Screening (All Selection Methods) — Key Metrics

Table 4: BM25 screening (18 selection methods, MAX_QUERIES=200): compact key metrics (portrait). Higher is better for MRR/P@1/nDCG; lower is better for ranking_error.

method	n_queries	mrr_l1	P@1_L1	nDCG@10	ranking_error_l1	mrr_l2	P@1_L2	ranking_error_l2
D_tfidf_80	200.0	0.9443	0.92	0.8495	0.365	0.3714	0.34	514.54
D_tfidf_160	200.0	0.9364	0.9	0.8458	0.485	0.3793	0.355	514.715
D_tfidf_120	200.0	0.9297	0.89	0.8386	0.335	0.3644	0.33	514.93
B_medium_k10	200.0	0.9247	0.885	0.8369	0.54	0.3793	0.36	514.765
B_medium_k20	200.0	0.9118	0.86	0.8303	0.47	0.3757	0.345	514.495
C_best_sent_n20	200.0	0.9246	0.875	0.82	0.24	0.3606	0.315	514.375
C_best_sent_n15	200.0	0.9309	0.885	0.8116	0.23	0.3703	0.33	514.355
B_medium_k5	200.0	0.9191	0.875	0.8014	0.445	0.3439	0.3	515.505
B_longest_k20	200.0	0.9101	0.86	0.7959	0.74	0.3708	0.335	514.825
C_best_sent_n10	200.0	0.9126	0.86	0.7941	0.375	0.3579	0.32	514.51
E_dense_extractive	200.0	0.9108	0.85	0.7873	0.375	0.3482	0.315	514.93
B_longest_k10	200.0	0.9071	0.845	0.7773	0.53	0.3806	0.355	515.52
B_longest_k5	200.0	0.8981	0.83	0.7559	0.51	0.3764	0.35	516.235
B_balanced_k20	200.0	0.9056	0.85	0.7451	0.66	0.3869	0.37	516.065
B_balanced_k10	200.0	0.8917	0.82	0.7404	0.71	0.3756	0.35	516.2
E_llm_qwen_test50	25.0	0.96	0.92	0.7379	0.08	0.718	0.56	1.0
B_balanced_k5	200.0	0.8687	0.79	0.7058	0.73	0.3677	0.335	516.035
A_baseline_all	200.0	0.8146	0.71	0.6645	0.925	0.3794	0.36	516.28

A.2 Top-5 Selection Methods × Models (35 Runs) — Key Metrics

Table 5: Top-5 selection methods × ranking models (35 runs, 917 queries): compact key metrics (portrait). Same conventions as Table 4.

method	model	n_queries	mrr_l1	P@1_L1	nDCG@10	ranking_error_l1	mrr_l2	ranking_error_l2
B_medium_k20	dense_mpnet	917.0	0.922	0.8822	0.8393	1.0796	0.3543	507.3402
B_medium_k10	dense_mpnet	917.0	0.9215	0.8811	0.8388	1.0807	0.3543	507.3391
D_tfidf_160	hybrid_alpha0.5	917.0	0.9219	0.879	0.8348	0.8212	0.3702	505.6619
D_tfidf_120	hybrid_alpha0.5	917.0	0.9256	0.8877	0.8332	0.8059	0.3714	505.4133
D_tfidf_80	hybrid_alpha0.5	917.0	0.9213	0.8757	0.832	0.7176	0.3676	505.6772
D_tfidf_120	dense_rerank_top50	917.0	0.9103	0.8626	0.8302	0.7415	0.3622	28.6914
D_tfidf_120	dense_allMiniLM	917.0	0.9164	0.8779	0.8292	0.9389	0.3599	506.7863
D_tfidf_160	dense_allMiniLM	917.0	0.917	0.879	0.8278	0.9673	0.3617	506.5202
D_tfidf_160	dense_rerank_top50	917.0	0.9126	0.8648	0.8273	0.7699	0.3571	28.7612
D_tfidf_80	dense_allMiniLM	917.0	0.9202	0.8822	0.8269	0.6587	0.3584	506.3675
D_tfidf_80	dense_rerank_top50	917.0	0.9149	0.867	0.8268	0.6467	0.3641	28.6227
D_tfidf_160	dense_multiqa	917.0	0.9114	0.867	0.8228	0.7132	0.3673	505.7775
B_medium_k10	dense_multiqa	917.0	0.9124	0.8713	0.8227	1.0098	0.3549	507.0752
B_medium_k10	hybrid_alpha0.5	917.0	0.9067	0.8593	0.8217	1.0087	0.357	507.1799
B_medium_k20	dense_multiqa	917.0	0.912	0.8702	0.821	0.9924	0.3561	507.1243
D_tfidf_120	dense_multiqa	917.0	0.915	0.8713	0.8204	0.7884	0.3617	506.1559

Continued on next page

method	model	n_queries	mrr_l1	P@1_L1	nDCG@10	ranking_error_l1	mrr_l2	ranking_error_l2
B_medium_k20	tfidf	917.0	0.9106	0.8593	0.8201	0.6805	0.3604	505.9836
B_medium_k10	tfidf	917.0	0.9092	0.8615	0.8195	0.6423	0.3534	505.7732
B_medium_k10	dense_allMiniLM	917.0	0.9069	0.8593	0.8188	0.9149	0.3491	507.6892
B_medium_k20	dense_allMiniLM	917.0	0.9069	0.8593	0.8188	0.9149	0.3491	507.6892
D_tfidf_160	bm25	917.0	0.9223	0.8811	0.8172	0.6041	0.3726	505.108
B_medium_k20	hybrid_alpha0.5	917.0	0.9097	0.8626	0.8172	0.9029	0.3569	507.0611
D_tfidf_80	dense_multiqa	917.0	0.9013	0.8441	0.8149	0.7077	0.3597	506.2432
D_tfidf_160	dense_mpnet	917.0	0.9001	0.8484	0.812	0.8157	0.3513	506.8975
D_tfidf_120	dense_mpnet	917.0	0.9036	0.8517	0.8116	0.7415	0.3547	507.6489
D_tfidf_160	tfidf	917.0	0.9178	0.8735	0.8115	0.7426	0.368	505.1778
D_tfidf_120	bm25	917.0	0.9183	0.8746	0.8108	0.5202	0.3686	505.0665
D_tfidf_80	bm25	917.0	0.9127	0.8648	0.8094	0.5256	0.3603	505.5976
D_tfidf_80	dense_mpnet	917.0	0.906	0.8561	0.8093	0.6532	0.352	507.2279
D_tfidf_120	tfidf	917.0	0.9122	0.8604	0.804	0.626	0.365	505.4667
B_medium_k10	dense_rerank_top50	917.0	0.8901	0.8386	0.8009	0.9629	0.3297	29.2421
B_medium_k20	dense_rerank_top50	917.0	0.8887	0.8353	0.8004	0.9618	0.3301	29.2323
D_tfidf_80	tfidf	917.0	0.9062	0.8561	0.7974	0.6925	0.3633	505.9804
B_medium_k10	bm25	917.0	0.8996	0.8441	0.7939	0.6881	0.3516	506.9084
B_medium_k20	bm25	917.0	0.9041	0.8517	0.7924	0.5616	0.3651	506.4079